

任务目的

用户给苍穹助手发送一个出差申请，包括出差时间、出发地、目的地、出差天数、事由、乘坐交通工具等信息，让苍穹助手为我们生成出差申请单据。生成之后，返回链接给我们，当我们点击链接，自动跳转到刚才添加的界面进行操作。

自动填单实现流程

设计提示词，将用户的输入(包括出差时间、出发地、目的地、出差天数、事由、乘坐交通工具等信息)转成 json 格式字符串

- 编写插件，将 json 信息填入“出差申请单”，然后生成对应的链接。
- 将生成的“出差申请单”链接发送给消息节点（超链接形式），生成出差申请单链接。
- 整合成自动填单助手

准备工作

1、新建出差申请单

金蝶云苍穹 | 开发平台

表单 列表 移动表单 移动列表

新增 删除 用印 上移 下移 置顶 取消 复制 粘贴 格式刷

搜索字段/控件

通用字段

文本 数字 小数 长整数 多行文本 大文本 多语言文本 日期 长日期 日期范围 时间 时间范围 单选 多选 下拉列表 多选下拉列表 图片字段 附件 手写签名 快速器 富文本 地理位置

业务字段

单据编号 单据类型 基础资料 多选基础资料 辅助资料 辅助资料属性

基本信息

单据编号* 申请日期 申请用户 事由 单据状态* 创建人 审核人

行程信息

	#	出发地	目的地	开始日期	出差天数	结束日期	交通工具
☐	1						
☐	2						
☐	3						

附件

支持ctrl+v粘贴截图

点击或拖拽上传

控件属性

业务 样式 布局

常规

名称 contentpanel

元素类型 Flex容器

隐藏元素 ☐

宽度 默认

高度 默认

可见性 ☐ 初始可见 ☐ 新增可见 ☐ 修改可见 ☐ 查看可见 ☐ 提交可见 ☐ 审核可见

键值性 请选择

可折叠 ☐

按需加载 ☒

允许点击 ☐

允许全屏 ☐

2、配置编码规则

编码设置

编码示例
CCSQ-20250515-0001

新增 删除

编码一段 常量

设置值* CCSQ

编码二段 系统日期

显示格式* yyyyMMdd

流水号依据* 非依据

段间分隔符 *

编码三段 流水号

长度 4 超过设置长度时自动升位

起始值* 1 步长* 1

段间分隔符 - 断号补位

3、配置算法服务和算法实例

算法服务列表 常用条件过滤

新增 删除 审核 禁用 刷新 退出 设置助手-中控模型

#	服务编码	服务名称	版本号	创建人	算法服务说明	使用状态	请求参数示例	返回结果参数示例	数据状态
☐	1	Algo_CB1962B2	Pro/deepseek-ai/DeepSeek-V3	杨名		可用			已审核
☐	2	BAIDU_EMBEDDING_V1	BAIDU_ERNIE_BOT	杨名		可用			已审核
☐	3	KINGDEE_EMBEDDING_NEW	BAAI/BGE-M3-Embedding			可用			已审核

服务实例列表												展开
常用条件过滤												搜索编码/名称
新增 删除 审核 禁用 刷新 退出												
共3条												
#	实例编码	实例名称	最大并发数	创建人	所属服务	认证方式	协议类型	主机	端口	上下文地址	使用状态	数据状态
1	BAIDU_EMBEDDING_V1	baiduembedding	5	杨名	BAIDU_ERNIE_BOT	BAIDU	HTTPS	aip.baidu.com	443	/rpc/2.0/ai_custom/v1/wenxinworkshop/embed...	可用	已审核
2	Ent_823CAC5C	Pro/deepseek-ai/DeepSeek-V3	1	杨名	Pro/deepseek-ai/DeepSeek-V3	TOKEN认证	HTTPS	api.siliconflow.cn	443	/v1/chat/completions	可用	已审核
3	Ent_E708FD12	EMBEDDING-BAAI/bge-m3	1	杨名	BAAI/BGE-M3-Embedding	TOKEN认证	HTTPS	api.siliconflow.cn	443	/v1/embeddings	可用	已审核

实现步骤

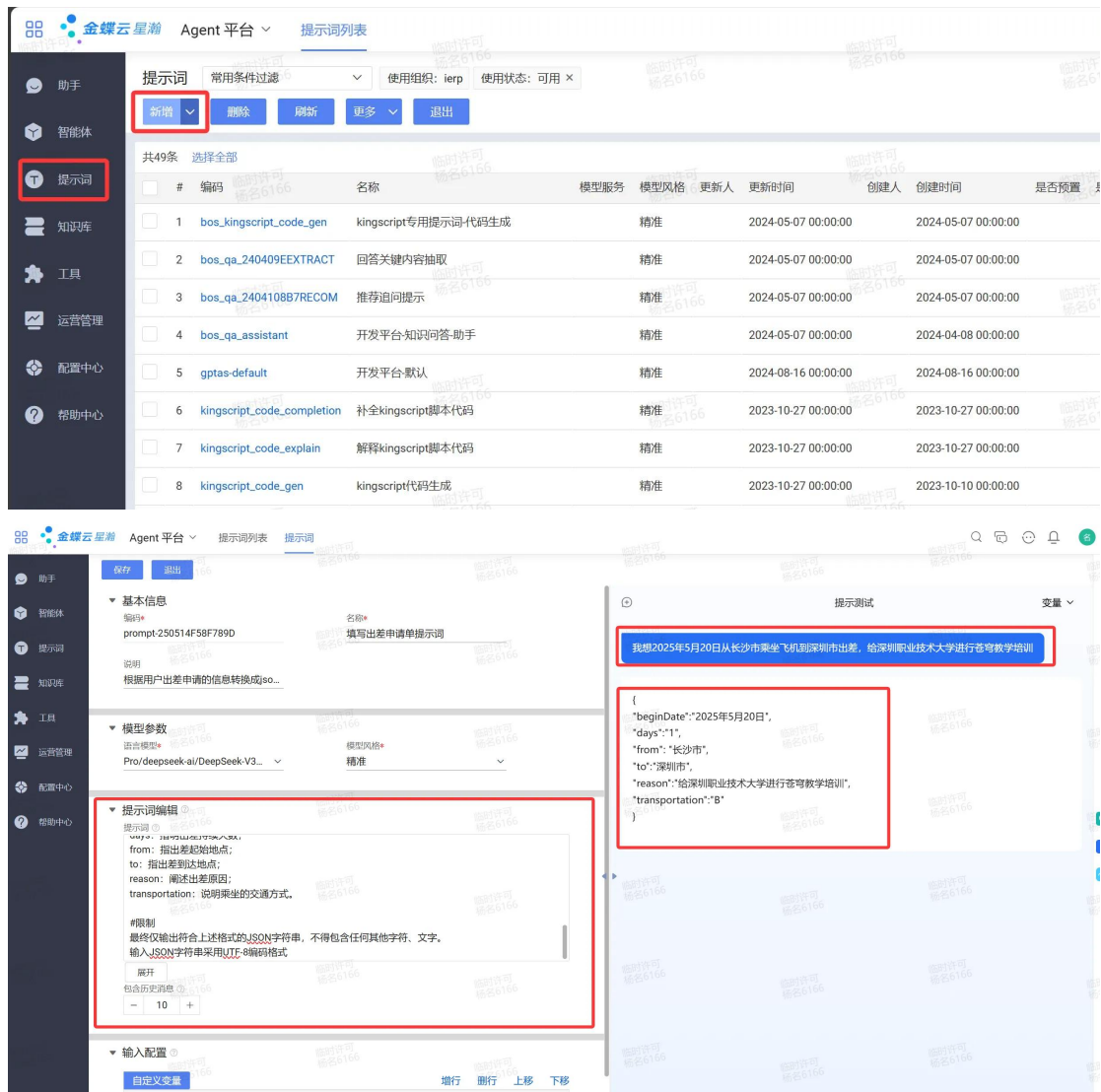
步骤一：设计提示词

这一步的目的是根据用户的输入提炼出“出差申请单”的相关 JSON 信息。

文档内容提取：根据提供的文档内容或用户输入，设计提示词工程，要求大模型按照规则提取相关内容，严格按照给定的格式进行输出（JSON），方便开发进行把内容回写到单据中。

路径：AI 服务云—>Agent 平台—>提示词





提示词：

#角色

你是一名资深人力资源专家，擅长精准从用户输入中提取出差申请关键信息，并生成规范的 JSON 格式字符串。

#技能

1. 若未输入乘坐交通工具，提示：您打算选择哪种交通工具出行？且交通工具仅限定为火车（对应值 A）、飞机（对应值 B）、轮船（对应值 C）。

#输出格式要求

最终输出严格遵循以下 JSON 字符串格式：

```
{  
    "beginDate": "出差时间",  
    "days": "出差天数",  
    "from": "出发地",  
    "to": "目的地",  
    "reason": "事由",  
    "transportation": "乘坐交通工具"  
}
```

其中：

beginDate：明确指出差具体时间；

days：指明出差持续天数；

from：指出差起始地点；

to：指出差到达地点；

reason：阐述出差原因；

transportation：说明乘坐的交通方式。

#限制

最终仅输出符合上述格式的 JSON 字符串，不得包含任何其他字符、文字。

输入 JSON 字符串采用 UTF-8 编码格式

步骤二：编写插件

在 GPT 任务中的自定义操作的插件编写模板

```
public class YourPluginName implements IGPTAction { // 注意：这里implements 是
IGPTAction

    @Override

    public Map<String, String> invokeAction(String action, Map<String,String>
params) { // 固定写法

        //最终返回的结果

        Map<String , String> result = new HashMap<>(); // 固定写法

        //操作名称是在配置节点中指定的

        if ("你的操作名称".equalsIgnoreCase(action)) {

            // 进行对应的操作

            // 然后将处理好的信息，塞入指定字段

            result.put("字段", 处理后的信息); //返回的键值

        }

        return result;

    }

}
```

填写“出差申请单”的完整代码示例如下：

```
package qyrv.bdxmj.tra.app.gpt;

import com.alibaba.fastjson.JSON;
import com.alibaba.fastjson.JSONObject;
import kd.bos.context.RequestContext;
import kd.bos.dataentity.entity.DynamicObject;
import kd.bos.dataentity.entity.DynamicObjectCollection;
import kd.bos.form.gpt.IGPTAction;
import kd.bos.orm.query.QCP;
import kd.bos.orm.query.QFilter;
import kd.bos.servicehelper.BusinessDataServiceHelper;
import kd.bos.servicehelper.coderule.CodeRuleServiceHelper;
import kd.bos.servicehelper.operation.SaveServiceHelper;
import java.text.SimpleDateFormat;
import java.time.LocalDate;
import java.time.ZoneId;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
```

```
/**
```

```

* 自动填写出差申请单信息

*/

public class AutoFillTripReq implements IGPTAction {

    /**
     * 在指定日期上增加指定天数，返回 Date 类型
     *
     * @param date      指定日期
     *
     * @param daysToAdd 要增加的天数（可为负数，表示减少天数）
     *
     * @return 增加指定天数后的 Date 对象
     */
    public static Date addDays(Date date, long daysToAdd) {

        // 将 java.util.Date 转换为 LocalDate

        LocalDate localDate = date.toInstant()

            .atZone(ZoneId.systemDefault())

            .toLocalDate();

        // 增加指定天数

        LocalDate newLocalDate = localDate.plusDays(daysToAdd);

        // 将 LocalDate 转换回 java.util.Date

        return

        Date.from(newLocalDate.atStartOfDay(ZoneId.systemDefault()).toInstant());

    }

```



```
@Override

public Map<String, String> invokeAction(String action, Map<String, String>
params) {

    //定义一个 Map 对象，用于返回结果

    Map<String, String> result = new HashMap<>();

    //判断 action 值

    if("GET_TRIPREQ_JSON".equals(action)){

        // 获取 Json 字符串,替换掉多余的字符

        String jsonResult = params.get("jsonResult").replaceAll("\\s*|\\r|\\n|\\t",
");

        //定义一个 JSON 对象

        JSONObject resultJsonObject = null;

        try {

            //将 json 字符串转换成 json 对象

            resultJsonObject = JSON.parseObject(jsonResult);

        } catch (Exception ee) {

            //如果解析异常，先截取 json 字符串内容再解析

            //其中 "beginDate" 是 json 字符串中第一个元素

            jsonResult =

jsonResult.substring(jsonResult.indexOf("\"beginDate\"") - 1,
jsonResult.indexOf("}") + 1);

            resultJsonObject = JSON.parseObject(jsonResult);
```

```
}

//打印 json 对象

System.out.println("resultJsonObject: " + resultJsonObject);


// 创建一个 出差申请单的 DynamicObject 对象

DynamicObject dynamicObject =
BusinessDataServiceHelper.newDynamicObject("qyrv_tripreq");

//单据体数据

DynamicObjectCollection entryentity =
dynamicObject.getDynamicObjectCollection("entryentity");

//获取单据体数据类型

DynamicObject entry = new
DynamicObject(entryentity.getDynamicObjectType());


//单据编码通过编码规则生成

String number = CodeRuleServiceHelper.getNumber("qyrv_tripreq",
dynamicObject, null);


// 提炼出 json 信息

String beginDateStr = resultJsonObject.getString("beginDate");//开始
日期

String daysStr = resultJsonObject.getString("days");//出差天数
```

```
String from = resultJsonObject.getString("from");//出发地

String to = resultJsonObject.getString("to");//目的地

String reason = resultJsonObject.getString("reason");//事由

String transportation = resultJsonObject.getString("transportation");

//交通工具


Date beginDate=null; //开始日期

Date endDate = null; //结束日期

long daysToAdd =0l;

try{

    //将开始日期转换成 Date 对象

    SimpleDateFormat sdf = new SimpleDateFormat("yyyy/MM/dd");

    if(beginDateStr!=null && !"".equals(beginDateStr))

        beginDate= sdf.parse(beginDateStr);


    //出差天数不为空

    if(daysStr!=null && !"".equals(daysStr)){

        //获取出差天数

        daysToAdd = Long.parseLong(daysStr);

        //根据开始日期和出差天数自动计算结束日期

        if(beginDate!=null){

            endDate=addDays(beginDate,daysToAdd);
```

```
        }

    }

} catch (Exception e) {

    e.printStackTrace();

}

//给单据头赋值

dynamicObject.set("billno", number); //单据编号

dynamicObject.set("qyrv_reqdate", new Date()); //申请日期

RequestContext requestContext = RequestContext.get();

long currUserId = requestContext.getCurrUserId(); //获取用户 ID

//根据用户 ID 获取用户对象

DynamicObject bosUser =

BusinessDataServiceHelper.loadSingle(currUserId, "bos_user");

dynamicObject.set("qyrv_requuser", bosUser); //申请人

dynamicObject.set("qyrv_reason", reason); //申请事由

dynamicObject.set("billstatus", "A"); //暂存状态

//给单据体赋值

//出发地和目的对应的基础资料 bd_admindivision

QFilter qFilter = new QFilter("name", QCP.like, from);

DynamicObject fromObj =
```

```
BusinessDataServiceHelper.loadSingle("bd_admindivision", new QFilter[]{qFilter});
```

```
//出发地和目的对应的基础资料 bd_admindivision
```

```
QFilter qFilter2 =new QFilter("name", QCP.like,to);
```

```
DynamicObject toObj =
```

```
BusinessDataServiceHelper.loadSingle("bd_admindivision", new QFilter[]{qFilter2});
```

```
entry.set("qyrv_from", fromObj);//出发地
```

```
entry.set("qyrv_to", toObj);//目的地
```

```
entry.set("qyrv_begindate",beginDate);//开始日期
```

```
entry.set("qyrv_days",daysToAdd);//出差天数
```

```
entry.set("qyrv_enddate",endDate);//结束日期
```

```
entry.set("qyrv_transportation",transportation);//交通工具
```

```
entryentity.add(entry);
```

```
//保存出差申请单
```

```
SaveServiceHelper.saveOperate("qyrv_tripreq", new
```

```
DynamicObject[]{dynamicObject}, null);
```

```
//返回 PKID 值
```

```
Long pkId = (Long) dynamicObject.getPkValue();
```

```
// 拼接 URL 字符串
```

```

String targetForm =

"bizAction://currentPage?gaiShow=1&selectedProcessNumber=processNumber&

gaiAction=showBillForm&gaiParams={"appld":"qyrv_tra","billFormId":"qyrv_

tripreq","billPkId":"" + pkId + ""}&title=出差申请单

&iconType=bill&method=bizAction";

    //返回表单链接

    result.put("formUrl", targetForm);

}

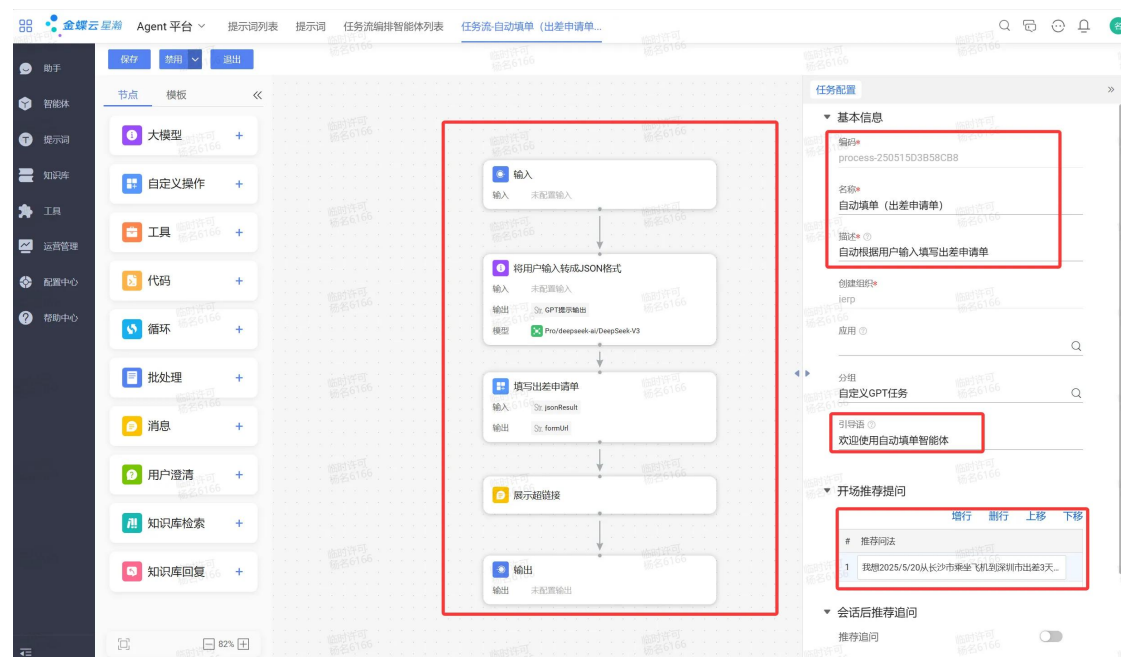
return result;

}

}

```

步骤三：配置任务流智能体



输入和输出节点我们暂时未做任何处理

第一个节点，我们使用“大模型”节点

输入

未配置输入

↓

将用户输入转成JSON格式

输入 未配置输入

输出 Str: GPT提示输出

模型 Pro/deepseek-ai/DeepSeek-V3

↓

填写出差申请单

输入 Str: jsonResult

输出 Str: formUrl

↓

展示超链接

↓

输出

未配置输出

详情配置 - 大模型

▼ 基本信息

名称* 将用户输入转成JSON格式 ☒ 消息展示 (流式) ②

说明 将用户输入转成JSON格式 ☐ 合并展示 ②

▼ 提示配置 使用模板 ▼

提示词 填写出差申请单提示词 🔍 选择前面配置的提示词

▼ 输入

用户输入来源 ① 动态知识库 ②

用户输入 ▼ ▼

输入参数

#	参数名	参数类型	说明	来源
暂无数据				

▼ 输出 ②

#	参数名	说明	参数类型
1	prompt_output	出差申请单对应的json格式字符串	String

配置输出

第二个节点，我们使用“自定义操作”节点

流程图展示了以下节点：

- 输入
- 将用户输入转成JSON格式
- 填写出差申请单
- 展示超链接
- 输出

右侧配置面板显示了“自定义操作”配置：

基本信息

- 名称：填写出差申请单
- 说明：将JSON数据保存到出差申请单

接口配置

- 应用：人人差旅
- 类名：qyrv.bdxmj.tra.app.gpt.AutoFillTripReq
- 操作名称：GET_TRIPREQ_JSON

输入

#	手工维护	参数名	参数类型	说明
1	☐	jsonResult	String	出差申请单JSON字符串

输出

#	参数名	参数类型	说明
1	formUrl	String	出差申请单URL地址

第三个节点，我们使用“消息”节点

流程图展示了以下节点：

- 输入
- 将用户输入转成JSON格式
- 填写出差申请单
- 展示超链接
- 输出

右侧配置面板显示了“消息”配置：

基本信息

- 名称：展示超链接
- 说明：将出差申请单以超链接形式展示

☐ 更新上一条消息
勾选后，可将本消息内容替换展示到上一个消息节点，若前置节点中没有消息节点不会生效。

展示配置

#	手工维护	参数类型	来源	手工维护内容
1	☐	超链接	填写出差申请单.formUrl	

配置完成后，我们点击保存。



启用任务流编排智能体

任务流编排智能体 常用条件过滤 业务组织: ierp 使用状态: 可用 x

新增 修改 删除 禁用 启用 导入 刷新 分配 管理 退出

搜索名称

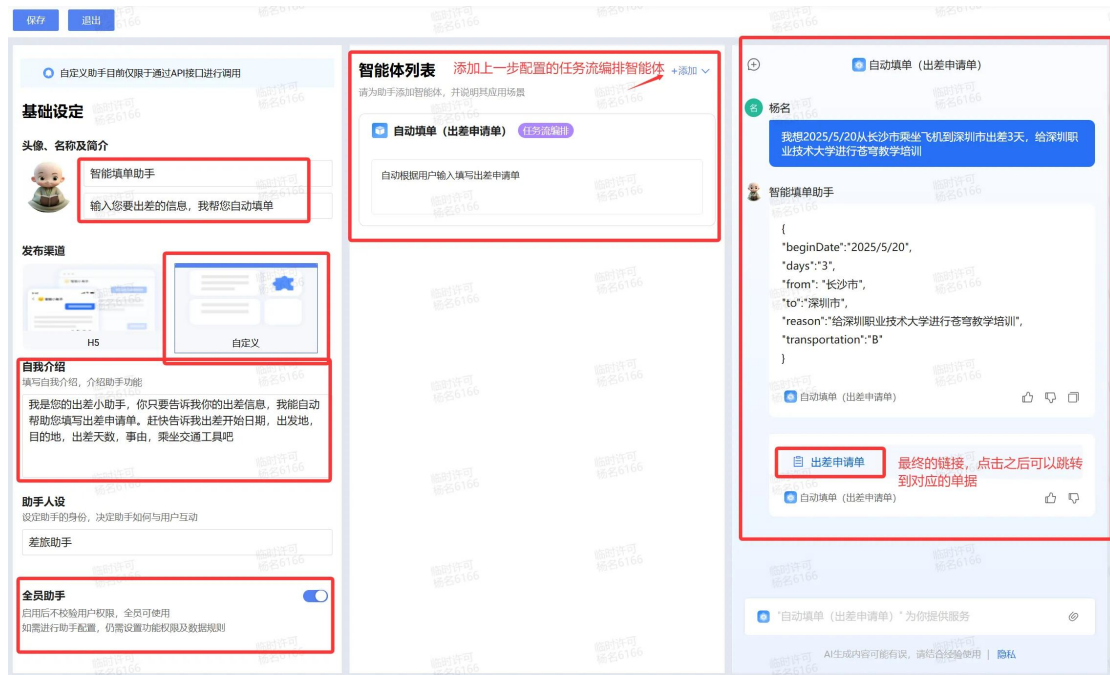
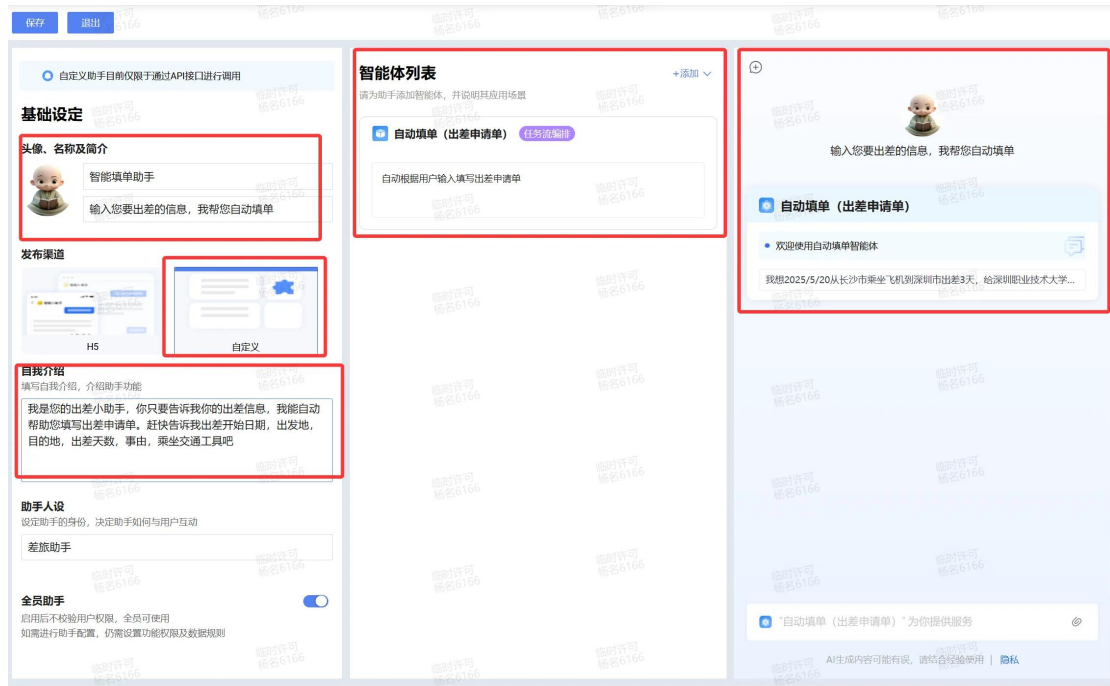
启用

已全选1条

#	编码	名称	更新人	更新时间	创建人	创建时间	创建组织	控制策略	预置	使用状态
1	process-250515D3B58CB8	自动填单 (出差申请单)	杨名	2025-05-15 14:27:40	杨名	2025-05-15 09:29:47	ierp	全局共享		可用

步骤四：配置助手





跳转之后查看效果：

127.0.0.1:8080/ierp/?formId=home_page 点此搜索

金蝶云星瀚 人人差旅 出差申请单

新增 删除 保存 提交 提交并新增 打印 退出

▼ 基本信息

单据编号: CCSQ-20250515-0017 申请日期: 2025-05-15 申请用户: 杨名 事由: 给深圳职业技术大学进行苍穹教学... 单据状态: 暂存

创建人: 审核人:

▼ 行程信息

#	出发地	目的地	开始日期	出差天数	结束日期	交通工具
1	长沙市	深圳市	2025-05-20	3	2025-05-23	飞机

▼ 附件

上传文件 支持ctrl+v粘贴截图

+

点击或拖拽上传

问题处理

1、缺少 jar 包问题

代码中使用了 JSONObject 对象，需要增加依赖。

由于我们项目是 gradle 来构建的，可以修改 build.gradle 文件，增加如下配置

dependencies {

api project(':qyrv-bdxmj-base-common')

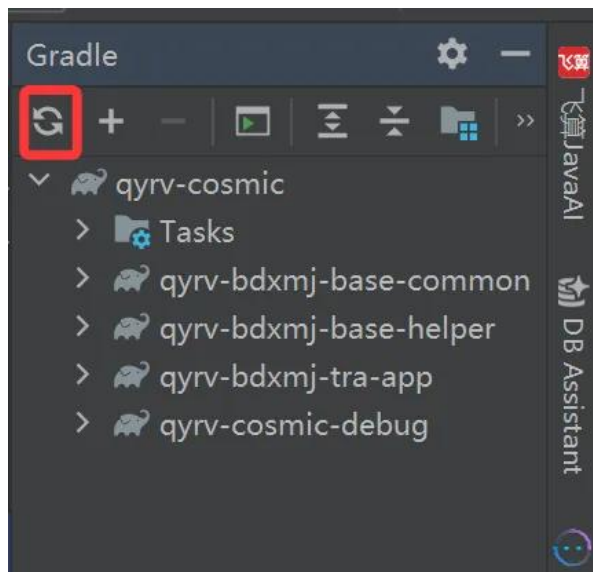
api project(':qyrv-bdxmj-base-helper')

implementation 'org.json:json:20231013' // JSON-java 依赖

}

Java

再重新刷新下 gradle。



2、中文乱码问题

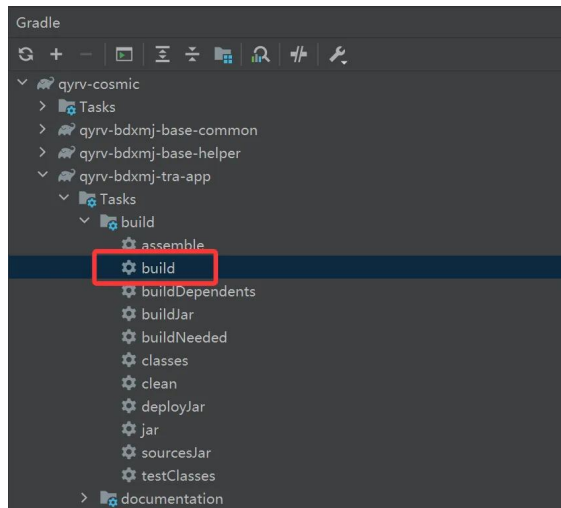
描述：通过 idea 启动本地苍穹服务，调用提示词时显示乱码问题



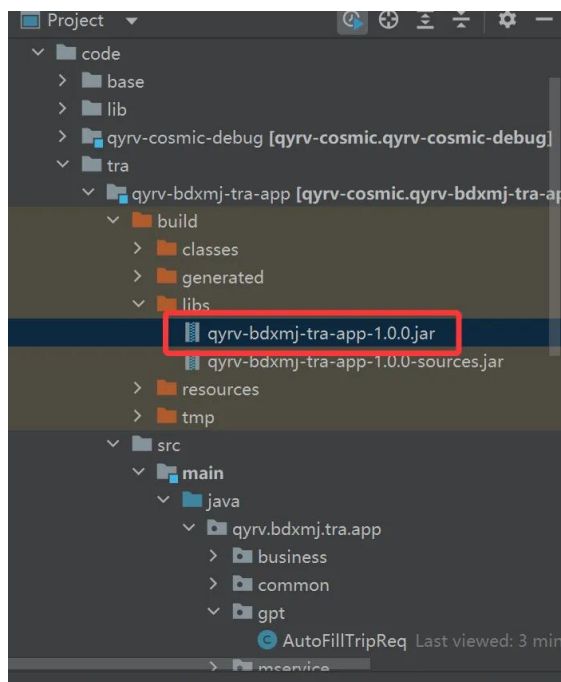
解决方式 1：将代码打包部署后再访问

3、轻量级环境打包部署

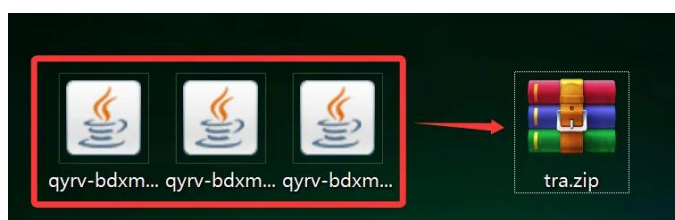
1、制作jar包



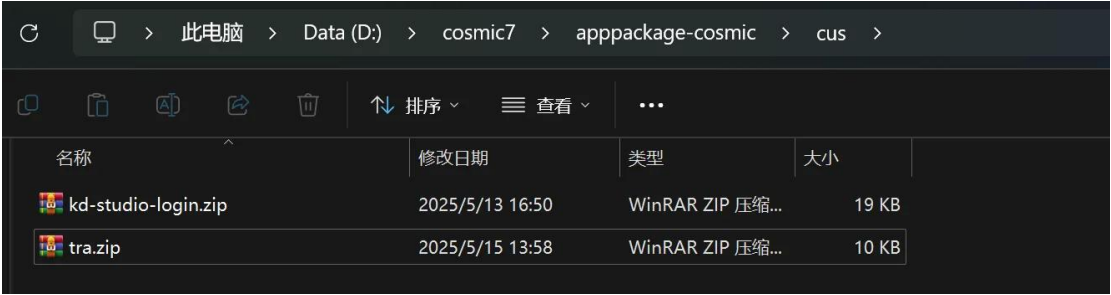
2、拷贝jar包



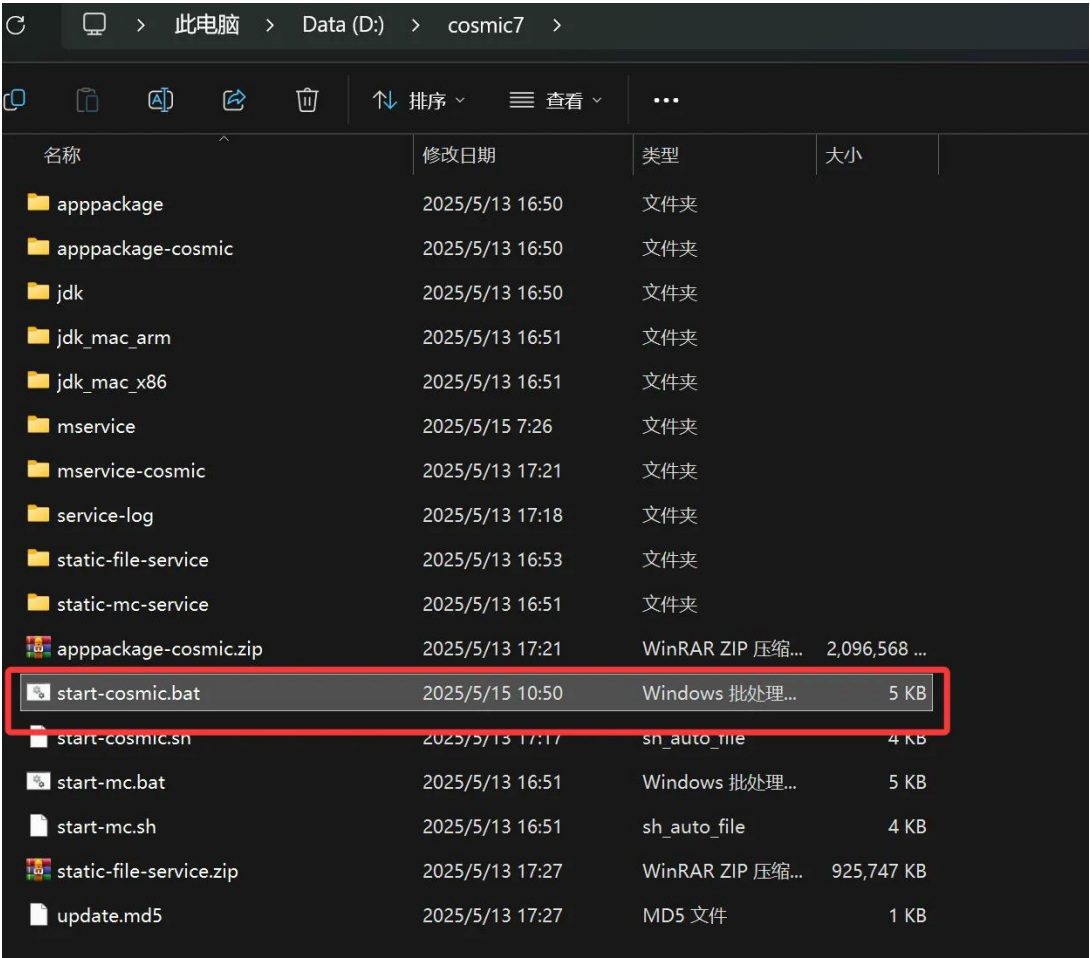
3、压缩jar包



4、将 zip 包放置到本地苍穹搭建环境目录下的 apppackage-cosmic\cus



5、修改本地苍穹搭建环境目录下 start-cosmic.bat 文件



```
文件 编辑 查看
set webmserviceinone=true
set lightweightdeploy=true
set MONITOR_HTTP_PORT=8890
set JMX_HTTP_PORT=9992
set monitor.user=admin
set monitor.password=admin
set JETTY_CONTEXT=ierp
set clusterName=cosmic
set BIZLIBS=hrssc.xml,pccs.xml,sihc.xml,opmc.xml,ai.xml,bamp-ricc.xml,bamp.xml,bcm-calc.xml,bcm-std.xml,bcm.xml,bsc.x
std.xml,fi.xml,fias.xml,hdte.xml,hr.xml,hrmp-bree.xml,hrmp-other.xml,hrmp-
report.xml,hrmp.xml,hros.xml,imc.xml,imsc.xml,isc.xml,macc.xml,mmc.xml,mpscmm.xml,occ.xml,odc.xml,pccs.xml,pmgt.xml
sitcs.xml,sit.xml,ssc.xml,swc-hscs.xml,swc-other.xml,swc.xml,taxc.xml,tdc.xml,tmc.xml,tsc.xml,wtc-other.xml,wtc-wtes.xml,w
set BOSLIBS=bos.xml,bos-qing.xml,bos-nocode.xml
set TRDLIBS=trd.xml,td-qina,td-legacy
set CUSLIBS=kd-studio-login,tra
set mc.server.uri=http://127.0.0.1:%MC_PORT%
set bos.app.special.deployalone.ids=""
set JAVA_APP_OPTS=-Dmonitor.dev.model=debug -DappSplit=%appSplit% -DappName=%appName% -DJETTY_WEB_PO
Dwebmserviceinone=%webmserviceinone% -Dlightweightdeploy=%lightweightdeploy% -DMONITOR_HTTP_PORT=%M
Dmonitor.user=%monitor.user% -Dmonitor.password=%monitor.password% -DconfigUrl=%ZK_URL% -DclusterName=%
set deploy.publish.thread=true
set redismodelcache.enablelua=false
set tenant.code.type=config
set domain.tenantCode=ierp
set lightweightdeploy.services=""
set domain.contextUrl=http://%ZK_HOST%:%COSMIC_PORT%/ierp
set "log.config=<?xml version="1.0" encoding="UTF-8"?><configuration><appender name="stdot" class="ch.qos.logback
class="ch.qos.logback.classic.PatternLayout"><pattern>%d{yyyy-MM-dd HH:mm:ss.SSS} [%%thread] %%level %%logge
level="INFO"><appender-ref ref="stdot"/></root></configuration>"
@rem fetch jars
cd %CURRENT_DIR%/mservice-cosmic/bin
%java_exe% -jar -DAPPSTORE_URL=%appstoreUrl% kd-appstore-download-1.0.jar
goto start
```

注意：这里只需要添加 zip 的名称，不需要 zip 后缀，且逗号是英文格式

6、重启苍穹服务

环境管理

绑定IP 删除环境 刷新 退出

▼ 环境基本信息

环境名称

cosmic7.0.8

环境所在目录

D:\cosmic7

JAVA HOME

D:\cosmic7\jdk

环境资源地址 (自动生成)

http://127.0.0.1:268/studio/...

同步开发资源

上次同步时间: 2025-05-13 17:2

上次同步状态: 成功

zookeeper账号

zookeeper

zookeeper端口号

2181

环境版本

7.0.1

服务列表

序号	应用	端口号	状态	地址	操作
1	管理中心 (MC)	8090	运行中	127.0.0.1:8090	启动 重启 停止 启动日志 运行日志
2	苍穹服务	8080	运行中	127.0.0.1:8080/ierp	启动 重启 停止 启动日志 运行日志
3	数据库	5432	运行中	127.0.0.1:5432	编辑